



Chinese Society of Aeronautics and Astronautics
& Beihang University
Chinese Journal of Aeronautics

cja@buaa.edu.cn
www.sciencedirect.com



Planax: An Efficient High-Fidelity Benchmarking Platform for Large-Scale UAV Swarm Cooperative Combat

KEYWORDS

Flight dynamics simulation;
Multi-agent reinforcement
learning;
Unmanned Aerial Vehicle
(UAV);
Air combat maneuvering;
Parallel simulation;
High-fidelity modeling;
Combat benchmark

Abstract Research on large-scale coordination for fixed-wing UAV swarms faces a critical bottleneck: balancing high-fidelity physical realism with computational efficiency at scale. Existing simulators either sacrifice physical accuracy for speed, or lack the parallelism required by modern multi-agent reinforcement learning (MARL). To bridge this gap, this study proposes Planax, a JAX-based high-fidelity parallel simulation framework. By leveraging XLA compilation and operator fusion, Planax integrates a nonlinear six-degree-of-freedom (6-DOF) flight dynamics model—driven by authentic F-16 aerodynamic data—directly into GPU-accelerated SIMD kernels. This design enables end-to-end device residence and eliminates CPU–GPU transfer overhead in the simulation–learning loop. In addition, a hierarchical reinforcement learning (HRL) scheme is implemented to facilitate complex cooperative maneuvers (or sophisticated combat strategies) across diverse mission horizons. Experiments show that Planax supports up to 10^8 parallel agents and achieves more than a $10\times$ speedup over the state-of-the-art NeuralPlane. Across the benchmark suite—single-control, formation, and combat—policies trained in Planax exhibit robust cooperation and competitive behaviors under physically grounded constraints. Planax is open-source and available at <https://anonymous.4open.science/r/Aeroplanax-2087>.

1. Introduction

In modern aerial warfare, the operational paradigm is shifting from single-platform engagement to large-scale collaborative combat of Unmanned Aerial Vehicle (UAV) swarms. Due to the high dynamics of air combat and complex mission requirements, Multi-Agent Reinforcement Learning (MARL) has emerged as a transformative approach to enable autonomous coordination, such as formation reconfiguration and cooperative target tracking^{1,2}.

However, the successful application of MARL in aerospace systems is severely constrained by a fundamental computational bottleneck: the conflict between physical fidelity and simulation efficiency. Traditional high-fidelity flight simulators, such as JS-

BSim³ and X-Plane⁴, rely on serial CPU execution, which becomes computationally prohibitive for MARL training that requires billions of interaction steps across thousands of parallel agents. Conversely, existing MARL benchmarks often simplify UAVs into point-mass particles, ignoring crucial aerodynamic constraints and inertial coupling, which leads to a significant “sim-to-real” gap.

To bridge this gap, this paper proposes Planax, a high-fidelity parallel simulation framework designed for massive-scale fixed-wing UAV swarms. Distinct from existing works, Planax does not merely port flight equations to GPUs; it leverages the XLA (Accelerated Linear Algebra) compiler and JAX-based operator fusion to integrate a rigorous nonlinear six-degree-of-freedom (6-DOF) flight dynamics model directly into optimized CUDA kernels. This architecture enables end-to-end device residence, effectively eliminating the costly CPU–GPU memory transfer overhead that plagues traditional simulation–learning loops. Furthermore, to ensure aerodynamic realism, coefficients are derived from NASA F-16 wind tunnel data rather than simplified linear approximations.

The main contributions of this work are summarized as follows:

Peer review under responsibility of Editorial Committee of CJA.



Production and hosting by Elsevier

1. **High-Fidelity Parallel Dynamics Engine:** This paper develops a JAX-accelerated 6-DOF solver that maintains bit-for-bit physical consistency with validated F-16 models while supporting the parallel simulation of up to 10^8 agents, significantly outperforming traditional CPU-based simulators.
2. **Native MARL-Oriented Infrastructure:** Planax provides a scalable simulation architecture that supports the parallel interaction of up to 10^8 agents. It optimizes GPU memory management to allow real-time simulation of large-scale swarms in adversarial environments, achieving a speedup of over 10 times compared to the state-of-the-art NeuralPlane.
3. **Validation in Complex Tactical Scenarios:** This paper demonstrates the framework's utility by training Hierarchical Reinforcement Learning (HRL) agents in diverse tasks, including multi-UAV formation flight and cooperative air combat. Experimental results validate that Planax provides a robust and scalable digital infrastructure for advancing MARL research in realistic aerial environments.

2. Related Work

2.1. RL for Fixed-Wing Flight Control and Long-Horizon Maneuvers

Reinforcement learning (RL) has increasingly been adopted for fixed-wing flight control due to its ability to handle nonlinear dynamics and complex operational envelopes. Policy-gradient methods such as PPO⁵ have been widely used for attitude and trajectory control, benefiting from stable on-policy optimization in continuous control settings^{6–9}. Beyond online model-free RL, offline RL and data-efficient learning have been explored to mitigate the high sample cost of flight-control training and enable learning from logged trajectories¹⁰. In addition, goal-conditioned formulations have been used to improve generalization across navigation, path-planning, and multi-goal scenarios^{11–13}. For more challenging maneuver generation and adversarial engagements, hierarchical RL and structured decision-making have been shown to improve exploration and long-horizon credit assignment¹⁴.

Despite these advances, flight-control RL remains constrained by safety, data efficiency, and sim-to-real transfer issues, particularly under extreme conditions and long-horizon tasks¹⁵. These constraints motivate the need for high-fidelity yet scalable simulators and standardized evaluation protocols, so that algorithmic progress can be assessed beyond one-off demonstrations.

2.2. Multi-Agent Learning in Multi-Aircraft Systems

Many aviation scenarios naturally require multi-agent reinforcement learning (MARL), including formation flight, cooperative surveillance, collision avoidance, and air-traffic-like coordination. Such problems exhibit partial observability, non-stationarity, and coupled dynamics, making coordination and robust learning difficult at scale. Representative MARL algorithms include value-decomposition methods (e.g., VDN, QMIX and its extensions)^{2,16,17}, centralized-training-decentralized-execution (CTDE) policy-gradient methods (e.g., MAPPO-style training)¹, as well as hybrid and hierarchical approaches¹⁴. Self-play is also widely used to reduce reliance on curated opponents and to improve competitive robustness¹⁸.

From a benchmarking perspective, classical MARL testbeds such as MPE¹⁹ and SMAC²⁰ focus mainly on low-dimensional or game-like settings. While they are valuable for algorithm prototyping, they do not capture the continuous-time, physics-consistent

constraints that dominate multi-aircraft systems. Recent aeronautics-oriented studies emphasize cooperative decision-making and communication constraints in more realistic settings^{21–23}, reinforcing the need for physically grounded MARL benchmarks.

2.3. Aircraft Simulation Platforms and Scalable Training Infrastructure

High-fidelity flight simulation has long relied on mature tools such as X-Plane⁴ and open-source systems such as ArduPilot²⁶ and JSBSim³. However, commercial simulators often lack headless, massively parallel execution, and research-oriented stacks may require significant engineering to integrate with modern RL pipelines²⁷. For learning-centric research, fixed-wing RL platforms such as QPlane²⁴ and LAG/CloseAirCombat-style environments²⁸ provide standardized interfaces but are typically limited in scalability.

To address the efficiency bottleneck, recent RL infrastructures leverage GPU acceleration and compilation-based systems. Isaac Gym demonstrates the benefits of GPU-accelerated physics for fast RL training²⁹, while JAX-based ecosystems highlight how JIT compilation and vectorization enable high-performance simulation and learning loops^{30–32}. NeuralPlane pioneered a GPU-accelerated, parallelizable fixed-wing benchmark with detailed flight dynamics, substantially improving simulation efficiency for RL studies²⁵. Building on these lines, Planax positions itself as a JAX-based platform that targets the multi-agent regime and aims to enable *high-fidelity* fixed-wing simulation with *efficient* parallel rollouts, thereby supporting scalable training and evaluation in realistic MARL settings.

To make the scope of Planax explicit and align with an engineering-oriented presentation style, Table 1 contrasts representative fixed-wing simulation stacks and learning-centric environments along several dimensions that directly determine MARL feasibility at scale: (i) *native* multi-agent rollouts, (ii) GPU/JIT acceleration for efficient headless execution, (iii) high-fidelity 6-DOF dynamics driven by aerodynamic data, (iv) standardized evaluation/visualization toolchain, and (v) explicit modeling of constraints (e.g., envelope / safety / mission constraints). In particular, Planax extends the GPU-accelerated fixed-wing benchmark line (e.g., NeuralPlane) toward the *multi-agent regime* by providing *native* large-scale interaction and an end-to-end benchmarking workflow, while preserving physically grounded dynamics.

2.4. High-Fidelity Nonlinear Dynamics and Efficient Training Platforms

A core obstacle in applying (multi-agent) reinforcement learning to fixed-wing aircraft is not merely the lack of game-style benchmarks, but the difficulty of simultaneously achieving (i) *physically credible* nonlinear flight dynamics and (ii) *training-time efficiency* at the scale required by modern policy optimization. Unlike many classical MARL testbeds that adopt kinematic point-mass abstractions, fixed-wing learning environments must respect rigid-body coupling, aerodynamic nonlinearity, actuator limits, and flight-envelope constraints; otherwise, learned behaviors can exploit non-physical loopholes and fail to transfer to realistic regimes.

On the modeling side, high-fidelity simulators (e.g., X-Plane, JSBSim) provide detailed flight dynamics and have been widely used in aerospace research, yet their CPU-centered execution and real-time-oriented design often become a bottleneck for RL, which typically requires millions to billions of interaction steps. On the infrastructure side, recent GPU-accelerated and compilation-based simulators demonstrate that large-scale, vectorized rollouts can dramatically improve RL training throughput. In particular, JIT compi-

Table 1 Comparison of representative fixed-wing simulation platforms and learning-centric environments. “Limited” indicates the capability is possible via additional wrappers or non-native extensions, but is not a primary design target.

Platform	Native MARL	GPU/JIT	6-DOF + aero data	Std. eval / viz	Explicit constraints	Typical limitation for large-scale MARL
X-Plane ⁴	No	No	Yes	Limited	Limited	Closed-source and real-time oriented; headless massive rollouts and RL integration require substantial engineering.
JSBSim ³	No	No	Yes	Limited	Limited	CPU-centered execution; throughput and parallelism become bottlenecks for modern policy optimization.
QPlane ²⁴	No	No	Yes (via backend)	Limited	Limited	Gym-style usability, but inherits backend performance/scalability limits for large-scale rollouts.
NeuralPlane ²⁵	Limited	Yes	Yes	Yes	Limited	GPU-accelerated fixed-wing benchmark, but not designed for <i>native</i> multi-agent interaction/training.
Planax (this work)	Yes	Yes	Yes	Yes	Yes	Unifies high-fidelity fixed-wing modeling with JAX acceleration and end-to-end MARL benchmarking at scale.

Table 2 Most essential symbols used in this paper.

Symbol	Meaning
$\mathcal{F}_E, \mathcal{F}_B, \mathcal{F}_W$	Earth/NED, body-fixed, and wind frames.
$\mathbf{x} \in \mathbb{R}^{13}$	13-D aircraft state (velocity, rates, position, quaternion).
$\mathbf{q}$	Unit quaternion for attitude (Earth→body).
$[T_{BE}]$	DCM from Earth to body (used for kinematics and gravity projection).
$\mathbf{u} = [\delta_a, \delta_e, \delta_r, \delta_T]$	Control inputs: aileron, elevator, rudder, throttle.
V, α, β	Airspeed magnitude, angle of attack, and sideslip.
$\mathbf{F}_B^{(A)}, \mathbf{M}_B^{(A)}$	Aerodynamic force/moment in body frame.
$C_L, C_D, C_Y, C_l, C_m, C_n$	Aerodynamic coefficients (forces/moments).
Δt	Integration step size (RK4).
$\mathcal{N}_i^i$	Neighbor set of agent i (range-limited observation).
e_t^i	Task error used for reward shaping (task-specific).
$r_t^i = 2e^{-ke_t^i} - 1$	Shaped reward; k is the shaping gain.
ε	Success threshold ($e_t^i < \varepsilon$ triggers REACHTARGET).
π_θ	Policy (actor) with parameters θ (PPO/IPPO/MAPPO variants).

lation and SIMD-style parallelism enable fusing dynamics integration and policy inference into efficient kernels, making long-horizon continuous-control training feasible under realistic physics.

Therefore, progress in fixed-wing MARL hinges on platforms that *jointly* provide: (i) validated nonlinear 6-DOF dynamics with physically grounded aerodynamics, and (ii) massively parallel, headless execution for efficient data collection. This perspective motivates Planax to unify high-fidelity fixed-wing modeling with JAX-based acceleration, so that multi-agent cooperation and adversarial interactions can be trained and evaluated under realistic constraints at scale.

Nomenclature

Table 2 lists only the most essential symbols.

3. Planax Framework: Physics-Consistent Modeling, XLA Acceleration, and Benchmark Suite

This section presents the core technical design of Planax, including the physics-consistent flight model, the XLA-accelerated parallel execution pipeline, and the benchmark interfaces used for learning and evaluation.

3.1. High-Fidelity Nonlinear Flight Dynamics Model

Motivation and scope. For multi-agent learning in fixed-wing aerial scenarios, *policy deployability* requires that the simulator strictly respects rigid-body coupling, actuator limits, and flight-envelope constraints; otherwise, learned behaviors may exploit non-physical

loopholes and fail to transfer to realistic regimes. Therefore, Planax adopts a validated nonlinear 6-DOF rigid-body aircraft model with quaternion attitude representation, and uses aerodynamic coefficients driven by NASA F-16 wind-tunnel data (implemented via a branch-free surrogate) to retain both physical credibility and SIMD efficiency under massive parallelism.

This section focuses on the *physical realism* of Planax. Each aircraft is modeled as a rigid body governed by nonlinear 6-DOF equations with quaternion attitude, and its aerodynamic coefficients are driven by NASA F-16 wind-tunnel data (via a branch-free surrogate). Supplementary expressions (DCM/kinematics matrices, component-wise EOM, and aerodynamic transformations) are deferred to Appendix A to keep the main text concise.

3.1.1. Reference Coordinate Systems

The motion of each aircraft is described using three standard frames: (i) Earth/NED frame $\mathcal{F}_E$, (ii) body-fixed frame $\mathcal{F}_B$ at the center of gravity, and (iii) wind frame $\mathcal{F}_W$ aligned with the airspeed vector. To avoid Euler-angle singularities, attitude is represented by a unit quaternion $\mathbf{q} = [q_0, q_x, q_y, q_z]^T$.

The aerodynamic angles are computed from body-frame velocities $\mathbf{v}_B = [u, v, w]^T$:

$$V = \|\mathbf{v}_B\|_2, \quad \alpha = \arctan 2(w, u), \quad \beta = \arcsin\left(\frac{v}{V}\right), \quad (1)$$

and the geometry conventions are illustrated in Fig. 1.

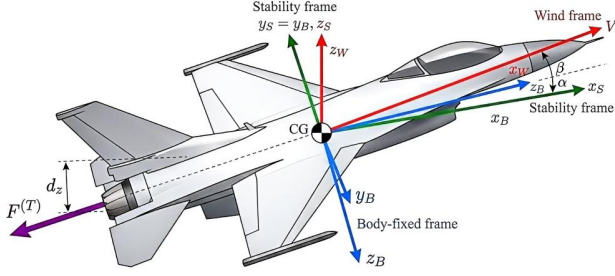


Fig. 1 Geometric definitions for the high-fidelity flight dynamics model utilized in Planax.

3.1.2. State Definition and Nonlinear 6-DOF Equations of Motion Each aircraft uses a 13-dimensional state:

$$\mathbf{x} = [u, v, w, p, q, r, x_E, y_E, z_E, q_0, q_x, q_y, q_z]^T, \quad (2)$$

where $\mathbf{v}_B = [u, v, w]^T$ is body-frame velocity, $\boldsymbol{\omega} = [p, q, r]^T$ is body-frame angular rate, $[x_E, y_E, z_E]^T$ is position in NED, and $\mathbf{q}$ is the attitude quaternion.

Rigid-body dynamics (core 6-DOF). The translational and rotational dynamics in $\mathcal{F}_B$ are:

$$\dot{\mathbf{v}}_B = \boldsymbol{\omega} \times \mathbf{v}_B + \frac{1}{m} (\mathbf{F}_B^{(A)} + \mathbf{F}_B^{(T)} + \mathbf{W}_B), \quad (3a)$$

$$\mathbf{I} \dot{\boldsymbol{\omega}} = \mathbf{M}_B^{(A)} + \mathbf{M}_B^{(T)} - \boldsymbol{\omega} \times (\mathbf{I} \boldsymbol{\omega}), \quad (3b)$$

where m is mass, $\mathbf{I}$ is the inertia tensor, $\mathbf{F}_B^{(A)}, \mathbf{M}_B^{(A)}$ are aerodynamic force/moment in body frame, $\mathbf{F}_B^{(T)}, \mathbf{M}_B^{(T)}$ are thrust force/moment, and $\mathbf{W}_B$ is weight projected into $\mathcal{F}_B$ via the DCM $[T_{BE}]$ (see Appendix A for the explicit DCM).

Kinematics. Position and attitude kinematics are integrated simultaneously:

$$\begin{bmatrix} \dot{x}_E \\ \dot{y}_E \\ \dot{z}_E \end{bmatrix} = [T_{BE}]^T \mathbf{v}_B, \quad \dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q}, \quad (4)$$

where $\boldsymbol{\Omega}(\boldsymbol{\omega})$ is the standard quaternion kinematics matrix (see Appendix A). The component-wise forms consistent with validated F-16 formulations are also provided in Appendix A.

3.1.3. High-Fidelity Aerodynamic and Thrust Models

Aerodynamic forces and moments. Planax computes aerodynamic forces/moments based on the wind-frame decomposition and transforms them to $\mathcal{F}_B$:

$$\mathbf{F}_B^{(A)} = [T_{BW}]^T \begin{bmatrix} -D \\ -Y \\ -L \end{bmatrix}, \quad (5a)$$

$$\mathbf{M}_B^{(A)} = \bar{q} S \begin{bmatrix} b C_l \\ \bar{c} C_m \\ b C_n \end{bmatrix}, \quad \bar{q} = \frac{1}{2} \rho V^2, \quad (5b)$$

where L, D, Y are lift/drag/side force in $\mathcal{F}_W$, $\bar{q}$ is dynamic pressure, ρ is air density, S is reference area, b is wingspan, and $\bar{c}$ is mean aerodynamic chord. The explicit $[T_{BW}]$ and the detailed coefficient-to-force relations are given in Appendix A.

Branch-free neural surrogate for aerodynamic coefficients. To preserve SIMD efficiency under massive parallelism, Planax uses a lightweight MLP to approximate F-16 aerodynamic coefficient tables:

$$\mathbf{C}_{aero} = [C_L, C_D, C_Y, C_l, C_m, C_n]^T = \text{MLP}_{aero}(\alpha, \beta, \boldsymbol{\delta}), \quad (6)$$

where $\boldsymbol{\delta} = [\delta_a, \delta_e, \delta_r, \delta_T]^T$ are control inputs. This surrogate avoids LUT-induced branch divergence and yields GPU-friendly dense linear algebra.

Thrust model. Thrust is modeled as a function of throttle δ_T , altitude h , and Mach M , and is applied along the body longitudinal axis with the appropriate thrust-line geometry; the full mapping and optional moment terms are summarized in Appendix A.

3.2. Parallel Solving and Efficient Architecture

Why Planax is fast — design target and bottleneck decomposition. For large-scale on-policy MARL, the training wall-clock time is dominated by the *environment stepping throughput*, rather than by the optimizer itself. Conventional “CPU simulator + GPU policy” pipelines incur three fundamental overheads: (i) *host-device transfers* of states/observations/actions each step, (ii) *Python-level dispatch and many small kernel launches* (fragmented simulation logic), and (iii) *SIMD inefficiency* caused by branch-heavy aerodynamic tables and heterogeneous control flow²⁹. Planax removes these overheads by expressing the entire per-step computation as a *single device-resident compiled program* and letting XLA perform whole-program optimization and fusion^{25,30}.

Per-step runtime model. The above statement can be made explicit by writing the end-to-end per-step latency as

$$t_{\text{step}} \approx t_{H2D} + t_{D2H} + n_{\text{launch}} t_{\text{launch}} + t_{\text{mem}} + t_{\text{compute}}, \quad (7)$$

where t_{H2D}/t_{D2H} are PCIe/NVLink transfers, n_{launch} is the number of GPU kernel launches per step, t_{launch} is the launch overhead, and $t_{\text{mem}}/t_{\text{compute}}$ are memory- and compute-time of the executed kernels. Planax targets $t_{H2D} \approx t_{D2H} \approx 0$ and **minimizes** n_{launch} by compiling simulation, observation, reward, and termination into fused kernels²⁹.

3.2.1. Step function as a pure, stateless program (enabling whole-program compilation)

A prerequisite for compiler-level fusion is to represent the environment step as a *pure function* with explicit inputs/outputs:

$$(\mathbf{X}_{t+1}, \mathbf{O}_{t+1}, \mathbf{R}_t, \mathbf{d}_t) = \text{Step}(\mathbf{X}_t, \mathbf{U}_t, \boldsymbol{\xi}_t), \quad (8)$$

where $\mathbf{X}_t \in \mathbb{R}^{N \times 13}$ is the batched aircraft state, $\mathbf{U}_t \in \mathbb{R}^{N \times 4}$ is the batched control, $\boldsymbol{\xi}_t$ denotes RNG seeds / scenario randomness, and $\mathbf{O}_t, \mathbf{R}_t, \mathbf{d}_t$ are batched observations, rewards, and termination flags. In Planax, Eq. (8) contains: (i) aerodynamic coefficient inference, (ii) 6-DOF derivative evaluation, (iii) RK4 integration, (iv) observation mapping, reward shaping, and termination checks. This “single-entry single-exit” design is critical: it turns the entire environment step into a single optimizable computation graph, instead of many small Python functions.

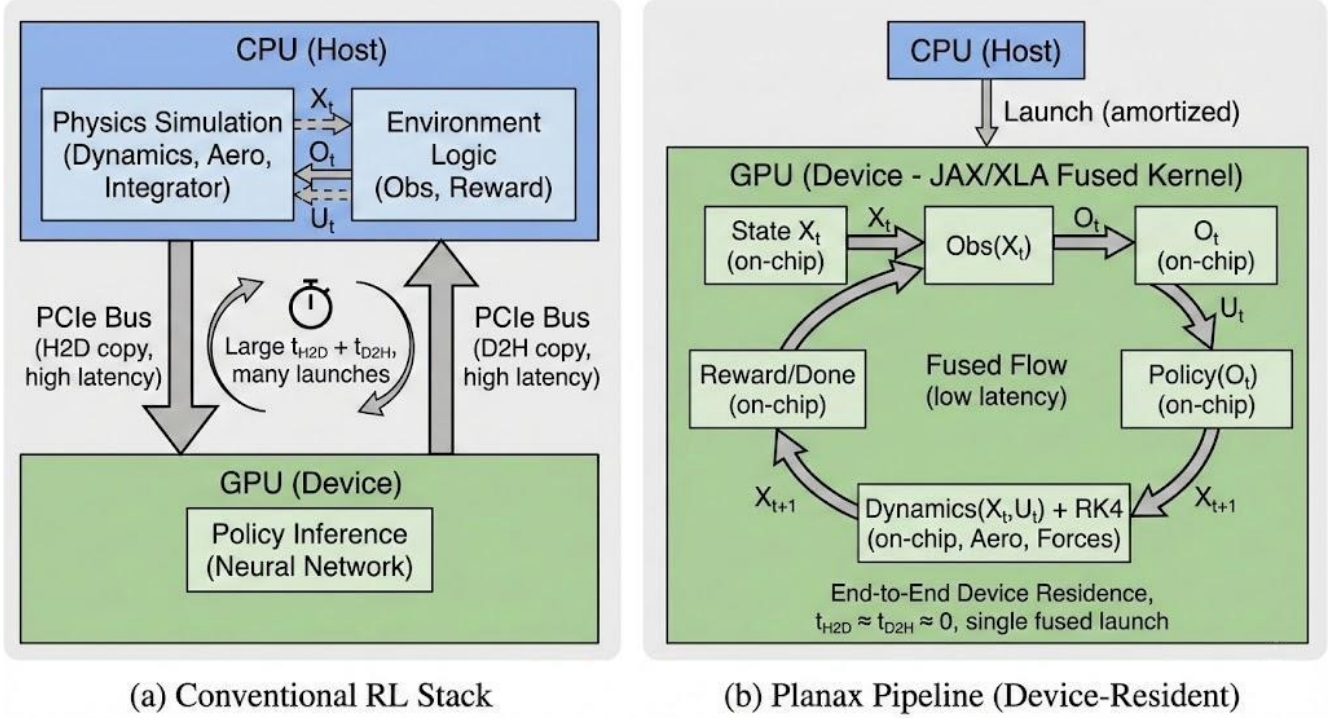


Fig. 2 End-to-end throughput bottlenecks and the Planax solution. (a) **Conventional RL stack**: fragmented CPU simulation and frequent host-device transfers lead to large t_{H2D}/t_{D2H} and high n_{launch} . (b) **Planax**: the per-step computation is expressed as a pure JAX function and compiled by XLA into fused GPU kernels, enabling end-to-end device residence and launch amortization.

Algorithm 1 Compilation pattern for massively parallel rollouts in Planax

Require: Single-aircraft dynamics $\text{Dyn}(\cdot)$, observation/reward/termination $\text{ObsRewDone}(\cdot)$

- 1: Define single-aircraft step: $\text{step}_1(x, u, \xi) \rightarrow (x', o', r, d)$
- 2: Vectorize over agents: $\text{Step} \leftarrow \text{vmap}(\text{step}_1)$
- 3: Unroll over time: $\text{Rollout}(X_0, \Xi_{1:T}) \leftarrow \text{scan}(\text{Step}, X_0, \Xi_{1:T})$
- 4: Compile once: $\text{Rollout}^{\text{jit}} \leftarrow \text{jit}(\text{Rollout})$
- 5: Execute on GPU repeatedly with different RNG/seeds/batches

3.2.2. *Batch parallelism and time unrolling: vmap and scan as the core scaling primitives*

Agent-axis vectorization. The agent population axis is parallelized by applying a vectorized map over aircraft instances. Conceptually, for a single-aircraft step function $\text{step}_1(\cdot)$, the batched step is

$$\text{Step}(\mathbf{X}_t, \mathbf{U}_t, \xi_t) = \text{vmap}(\text{step}_1)(\mathbf{X}_t, \mathbf{U}_t, \xi_t), \quad (9)$$

which exposes SIMD-style data parallelism to the compiler.

Horizon-axis unrolling (eliminating Python loops). Long-horizon rollouts (e.g., $T=10,000$ steps) can be dominated by Python overhead and repeated kernel launches if implemented as a Python for-loop. Planax therefore unrolls the temporal loop using a staged loop primitive and compiles the entire rollout into a small number of device programs:

$$(\mathbf{X}_{1:T}, \mathbf{O}_{1:T}, \mathbf{R}_{1:T}, \mathbf{d}_{1:T}) = \text{scan}(\text{Step}, \mathbf{X}_0, \xi_{1:T}), \quad (10)$$

The complete compilation workflow, enabling the transition from a single-agent step to a massively parallelized and JIT-compiled rollout, is summarized in Algorithm 1. so that time stepping becomes part of the compiled graph and kernel launches are amortized.

Algorithm 2 Fused parallel state propagation (Euler and RK4 Integration)

Require: Batched states $\mathbf{X}_t \in \mathbb{R}^{N \times 13}$, controls $\mathbf{U}_t \in \mathbb{R}^{N \times 4}$, step size Δt

Ensure: Updated states $\mathbf{X}_{t+1}$

- 1: Define vectorized dynamics: $f_{\text{vmap}} \leftarrow \text{vmap}(f_{6\text{DOF}}, \text{in_axes} = (0, 0))$
- 2: **Select Integration Scheme:**
- 3: **if** scheme is **Euler** (Fastest) **then**
- 4: $\dot{\mathbf{X}}_t = f_{\text{vmap}}(\mathbf{X}_t, \mathbf{U}_t)$
- 5: $\mathbf{X}_{t+1} = \mathbf{X}_t + \Delta t \cdot \dot{\mathbf{X}}_t$
- 6: **else if** scheme is **RK4** (High-Fidelity) **then**
- 7: $\mathbf{k}_1 = f_{\text{vmap}}(\mathbf{X}_t, \mathbf{U}_t)$
- 8: $\mathbf{k}_2 = f_{\text{vmap}}(\mathbf{X}_t + 0.5\Delta t \mathbf{k}_1, \mathbf{U}_t)$
- 9: $\mathbf{k}_3 = f_{\text{vmap}}(\mathbf{X}_t + 0.5\Delta t \mathbf{k}_2, \mathbf{U}_t)$
- 10: $\mathbf{k}_4 = f_{\text{vmap}}(\mathbf{X}_t + \Delta t \mathbf{k}_3, \mathbf{U}_t)$
- 11: $\mathbf{X}_{t+1} = \mathbf{X}_t + \frac{\Delta t}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$
- 12: **end if**
- 13: **return** $\mathbf{X}_{t+1}$ {Fully device-resident and XLA-fused}

3.2.3. *XLA kernel fusion: from fragmented operators to fused device kernels*

Aero + 6-DOF + RK4 as one fused device program. Traditional CPU-based simulators (e.g., JSBSim) execute the dynamics engine via modular, fragmented function calls, which amplifies dispatch overhead and intermediate memory traffic³. In Planax, the entire nonlinear step pipeline—aerodynamic coefficient inference, 6-DOF derivative evaluation, and RK4 integration—is expressed as a single JAX program and compiled into a fused GPU kernel by XLA^{25,30}. At the HLO level, XLA fuses producer-consumer chains to reduce global memory reads/writes, so runtime is less limited by memory

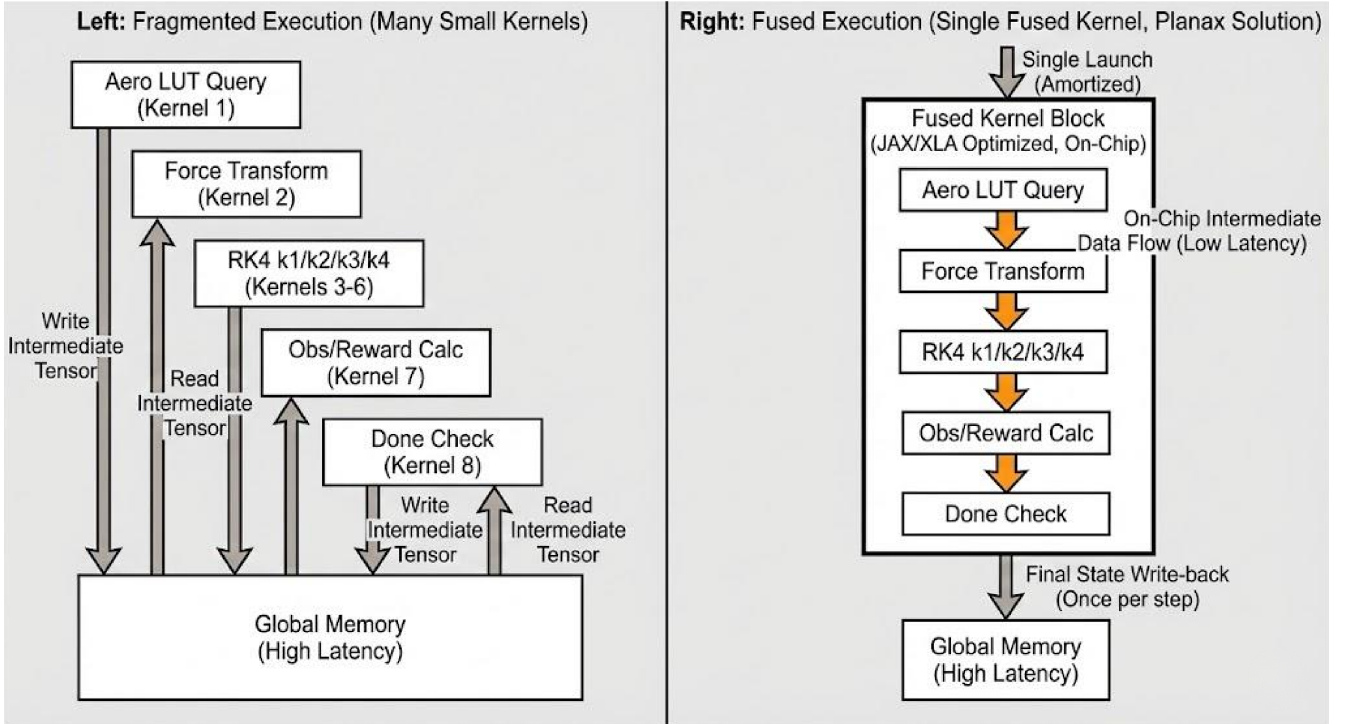


Fig. 3 Illustration of why kernel fusion improves efficiency. **Left:** a fragmented implementation launches many kernels and writes intermediate tensors to global memory repeatedly. **Right:** Planax compiles the step as a fused program, reducing launch overhead and global-memory traffic.

bandwidth²⁹.

What gets fused in Planax. Within each step, Planax evaluates the neural aerodynamic surrogate, the rigid-body dynamics, and the numerical integrator. To balance computational speed and physical fidelity, the framework supports both explicit Euler integration (for maximum throughput during initial training) and 4th-order Runge-Kutta (RK4) integration (for high-fidelity evaluation). Algorithm 2 details this vectorized integration process. If executed as separate kernels, intermediate tensors (e.g., aerodynamic coefficients, forces/moments, and RK4 stages $\mathbf{k}_{1:4}$) would be written back to global memory multiple times, inflating t_{mem} in Eq. (7). By compiling the whole step, XLA can fuse producer-consumer chains and keep intermediates in registers/shared memory whenever possible, reducing global memory traffic and lowering n_{launch} .

Launch-count and transfer comparison. Eq. (7) shows that end-to-end throughput is jointly limited by host-device transfers ($t_{\text{H2D}}/t_{\text{D2H}}$), kernel-launch overhead ($n_{\text{launch}} t_{\text{launch}}$), and global-memory traffic (t_{mem}). Planax reduces these terms *simultaneously*: (i) end-to-end device residence makes $t_{\text{H2D}} \approx t_{\text{D2H}} \approx 0$ by producing states/observations/rewards/termination on device and consuming them directly in device-side inference (Fig. 2); (ii) whole-program compilation of the pure step function (Eq. (8)), together with scan-staged rollouts (Eq. (10)), amortizes launches and reduces n_{launch} ; (iii) XLA fusion keeps short-lived intermediates on-chip whenever possible, lowering t_{mem} during aerodynamic evaluation (Eq. (6)) and RK4 integration (Fig. 3).

3.2.4. SIMD efficiency: branch-free aerodynamics and uniform control flow

Modern GPUs execute threads in warps. When control flow diverges within a warp, the hardware serializes branch paths and the effective throughput decreases. This issue is especially pronounced

in branch-heavy implementations, such as multi-dimensional LUT queries and piecewise aerodynamic logic, which often introduce both divergence and irregular memory access patterns^{25,29}.

To mitigate this effect, Planax adopts two complementary design choices. First, aerodynamic coefficient tables are approximated by a lightweight MLP surrogate that maps (α, β, δ) to the coefficient vector. The main motivation is to obtain a continuous and well-defined coefficient field for arbitrary flight states, thereby avoiding discrete table indexing and interpolation corner cases during long-horizon rollouts. Second, termination checks and reward shaping are implemented using vectorized predicates^{25,33}, which reduces explicit branching in the step function and keeps the control flow more consistent across agents.

Together, these choices make the per-step computation largely uniform across the batched population. Most operations reduce to dense arithmetic with predictable execution patterns, which better matches the native strengths of GPUs. In addition, this formulation is favorable to JAX/XLA compilation and fusion³⁰, and thus supports efficient large-scale parallel rollouts in practice.

3.2.5. Memory footprint and scalability to extreme population sizes

Finally, large-scale MARL is feasible only if the memory footprint scales predictably with population size. Let b_{fp} denote bytes per floating-point element (e.g., $b_{\text{fp}}=4$ for FP32). A conservative per-step device footprint can be estimated as

$$\text{Mem} \approx N \cdot b_{\text{fp}} \cdot (d_x + d_o + d_u + d_{\text{aux}}), \quad (11)$$

where $d_x=13$ is the state dimension, d_o is the observation dimension (task-dependent), $d_u=4$ is the action dimension, and d_{aux} includes RK4 intermediates and bookkeeping tensors. Eq. (11) clarifies why (i) fusing kernels (reducing d_{aux} lifetime) and (ii) avoiding host-side copies are both necessary to push population sizes toward the 10^8 regime.

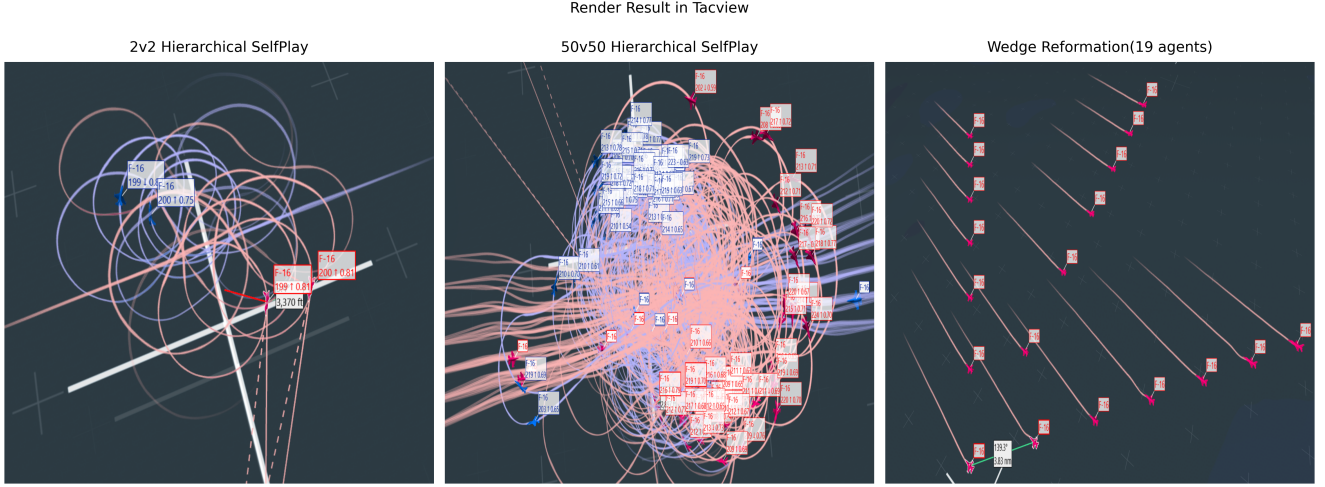


Fig. 4 Qualitative renderings of representative benchmark task scenarios.

Summary. Planax improves efficiency by reducing all major terms in the per-step runtime model (Eq. (7)). End-to-end device residence removes host–device transfers, whole-program compilation amortizes launches, and XLA fusion reduces global-memory traffic. Together with the physics-consistent 6-DOF model (Eqs. (3a)–(3b)) and the neural aerodynamic surrogate (Eq. (6)), the proposed design supports stepping up to 10^8 agents in the tested configuration, with the measured throughput reported in Fig. 7, exceeding NeuralPlane²⁵.

3.3. Task Scenario Design and Evaluation Protocol

3.3.1. Overview and benchmark suite

Overview. To turn the efficient pipeline in Section 3.2 into a reproducible learning and evaluation workflow, Planax defines a benchmark suite that covers *Single-Control*, *Formation*, and *Combat* tasks under a unified RL interface. Following common aerospace engineering practice, *mission-level* score sheets (e.g., success/win rates, survivors, health advantage, total return) are reported rather than only raw kinematic traces. Fig. 4 shows qualitative renderings of representative scenarios.

3.3.2. Unified interaction interface and tensorized obs/action convention

Multi-agent interaction interface. Consider an episode with N aircraft and horizon T . At time step t , each agent i receives an observation o_t^i , executes an action a_t^i , obtains a reward r_t^i , and produces a termination flag d_t^i . Planax exposes a unified tensorized interface $(\mathbf{O}_t, \mathbf{A}_t, \mathbf{R}_t, \mathbf{d}_t)$ for batched simulation and training.

Observation and action spaces. Unless otherwise stated, observations are composed of quantities typically available to avionics stacks (attitude, rates, velocity, relative geometry), and are normalized for stable optimization. Actions are continuous 4-D controls (throttle, elevator, aileron, rudder), scaled to $[-1, 1]$ for policy outputs. Table 3 summarizes the interface sizes used in this paper.

3.3.3. Single-Control task

Scenario description. Before studying large-scale coordination and competition, a fundamental requirement is to validate controllability under the high-fidelity aircraft model. The Single-Control benchmark focuses on tracking commanded flight states and executing

Table 3 Summary of observation/action interfaces for benchmark scenarios.

Scenario	Agents (N)	Horizon	Obs dim	Act dim
Formation	19 (e.g., wedge)	200 s	$13(1 + K)$, $K \leq N - 1$; max 247 (for $N = 19$)	4
Combat (self-play / vs baseline)	2-vs-2 to 50-vs-50	200 s	$13(1 + K)$, $K \leq N - 1$; max 52 ($N = 4$), 1300 ($N = 100$)	4

Notes: $\Delta t = 0.02$ s, hence $T = 200/0.02 = 10,000$ steps per episode. Each agent observes (i) its own normalized 13-D state and (ii) relative states of neighbors within a 10 km communication radius; K denotes the number of observed neighbors. Actions are continuous 4-D controls (throttle, elevator, aileron, rudder), scaled to $[-1, 1]$.

aggressive maneuvers. Two variants are considered: (i) Single-Control (Heading), which tracks a target heading while maintaining stable flight; (ii) Single-Control (Ultimate Goal), which requires continuous aggressive turning to track a moving target point.

Objective and error. The objective is to minimize the Euclidean distance between the current state vector x_t and the target state vector x_{target} :

$$e_t = \|x_t - x_{\text{target}}\|_2. \quad (12)$$

The reward is shaped by an exponential function of e_t , consistent with the formulation in Eq. (14).

3.3.4. Formation task

Task overview. The Formation task requires a team of N aircraft to maintain a predefined geometric pattern. Each aircraft is assigned a fixed *slot position* in the inertial frame. The learning objective is to minimize the tracking error to its own slot while respecting safety constraints.

Slot generation (formation template). A formation template is selected from several standard patterns (e.g., wedge, line, or diamond). Given a spacing parameter, the template produces N slot offsets in 3D space. A minimum separation constraint is enforced to avoid

overly close initialization. The slot positions remain fixed during an episode.

Per-agent tracking error. Let $p_t^i \in \mathbb{R}^3$ denote the inertial position of aircraft i at time t , and let $p_\star^i$ denote its assigned slot position. The per-step tracking error is defined as the Euclidean distance

$$e_t^i = \|p_t^i - p_\star^i\|_2. \quad (13)$$

Decentralized observation with top- K neighbors. Each agent observes only local information. The ego observation includes (i) slot-tracking errors in position, (ii) heading/speed deviations, and (iii) basic flight states such as angles and angular rates. In addition, the relative states of the closest K neighbors are appended. For each selected neighbor j , a compact relative feature vector is used: relative position (normalized), relative speed, and a normalized angle-of-approach term. Neighbors beyond a communication radius are masked as zeros.

Reward shaping. A smooth distance-based shaping reward is used. The distance to the assigned slot is normalized to kilometers,

$$r_{t,\text{track}}^i = \exp\left(-\frac{e_t^i}{1000}\right). \quad (14)$$

To avoid over-rewarding extremely small errors, an additional amplitude factor is applied when the distance is below 0.25 km. The final shaped reward is the product of the above terms.

Termination and event penalties. The episode terminates if an aircraft violates safety constraints (e.g., crash), or if the formation is considered unreachable. A large event-driven penalty is applied when an agent becomes inactive (e.g., crashed or shot down), using a fixed terminal value.

3.3.5. Combat task

Evaluation settings. The Combat benchmark models multi-agent aerial pursuit–evasion under the same flight dynamics and envelope constraints. Two standard settings are considered: (i) self-play, where both sides learn simultaneously; and (ii) vs baseline, where the learning side is trained/evaluated against a scripted or fixed-policy opponent.

Decentralized, range-limited observation. Combat follows the unified Dec-POMDP interface:

$$o_t^i = \text{normalize}\left(\left[x_t^i, \{x_t^j - x_t^i\}_{j \in \mathcal{N}_t^i}\right]\right), \quad (15)$$

where $\mathcal{N}_t^i$ includes both allies and opponents within the communication radius.

Task error and reward shaping. Let $d_{\text{TA}}(i)$ be the distance from aircraft i 's nose to the nearest opponent's tail, and $d_{\text{TA}}^{\min}$ be a safety margin. The per-step task error is

$$e_t^i = \max(0, d_{\text{TA}}(i) - d_{\text{TA}}^{\min}). \quad (16)$$

The shaped reward is then defined by

$$r_t^i = 2e^{-k}e_t^i - 1, \quad k = 3. \quad (17)$$

3.3.6. Unified termination constraints and statistical protocol

Per-step task error definitions. Table 4 summarizes the task error e_t^i definitions used by the exponential shaping across scenarios.

Table 4 Per-step task error e_t^i used by the exponential reward shaping.

Task	Definition of e_t^i
Single-Control	$\ x_t^i - x_{\text{target}}\ _2$
Formation	$\ p_t^i - p_t^{i,\text{des}}\ _2 \quad (i \geq 2)$
Combat	$\max(0, d_{\text{TA}}(i) - d_{\text{TA}}^{\min})$

Table 5 Termination conditions (identical across tasks).

Condition	Trigger	Terminal reward
EXTREMESTATE	AOA and AOS exceeding limit ranges	−200
HIGHSPEED	Mach > 3	−200
LOWALTITUDE	Altitude < 2500 ft	−200
LOWSPEED	Mach < 0.01	−200
OVERLOAD	$ G > 10$	−200
REACHTARGET	Task error $e_t^i < \varepsilon$	+200

Table 6 Default baselines for each evaluation setting.

Task	Setting	Baseline algorithm(s)
Single-Control	control	Standard PPO
Formation	cooperative	IPPO (Independent PPO)
Combat	self-play	Hierarchical MAPPO / End-to-End PPO
Combat	vs baseline	Hierarchical MAPPO / End-to-End PPO

Shared termination conditions (envelope/safety constraints). Table 5 lists termination conditions shared across tasks, ensuring trajectories remain within a physically meaningful flight envelope.

Mission-level score sheets and statistical reporting. Performance is summarized using mission-level metrics (success/win rates, survivors, health advantage, total return), averaged over multiple random seeds and reported as mean $\pm$ standard error. Together with the baseline library in Section 3.4, this protocol forms the closed training–testing–evaluation loop shown in Fig. 5.

3.4. Baseline Algorithm Library

3.4.1. Library overview and design principles

Design goal. The baseline library provides a set of *reproducible* reference learners that plug into the workflow in Fig. 5. The selection follows three principles: (i) covering both single-agent continuous control and multi-agent cooperation/competition; (ii) strict alignment with the unified tensorized interface (Table 3); and (iii) integrating structured inductive biases (CTDE, hierarchy, self-play) for improved trainability and interpretability in long-horizon non-stationary settings. Table 6 lists the default baseline algorithms used in each setting.

3.4.2. PPO: baseline for Single-Control

Proximal Policy Optimization (PPO) is adopted as the default baseline for Single-Control. It trains a stochastic actor $\pi_\theta(a|o)$ and a critic $V_\phi(o)$. The advantage $\hat{A}_t$ is computed by Generalized Advantage Estimation (GAE).

GAE advantage estimator. The temporal-difference residual is defined as

$$\delta_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t). \quad (18)$$

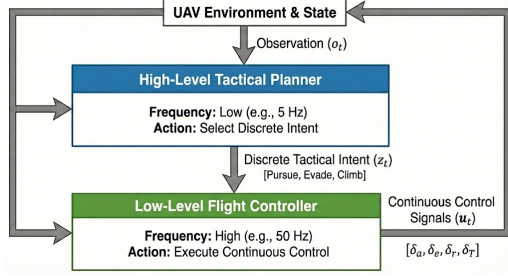


Fig. 6 Two-tier architecture used in Combat: MAPPO runs at the Decision-Making Level, while PPO outputs continuous low-level controls.

Decentralized actors (execution). Each agent i executes a local stochastic policy $\pi_{\theta_i}(a_t^i | o_t^i)$, consistent with decentralized deployment.

Centralized critic (training). Unlike PPO/IPPO, MAPPO replaces the local critic with a centralized value function that conditions on a global signal:

$$V_{\phi}(\cdot) \in \{V_{\phi}(s_t), V_{\phi}(\mathbf{o}_t)\}, \quad \mathbf{o}_t = [o_t^1, \dots, o_t^N]. \quad (26)$$

This centralized critic reduces value-estimation variance and yields more reliable advantage signals.

Policy update with centralized advantages. Advantage estimation follows the PPO definitions in Eqs. (18)–(19), but replaces $V_{\phi}(o_t)$ with the centralized critic $V_{\phi}(\cdot)$ in Eq. (26). Each actor is then updated by the same clipped objective in Eq. (21), while the centralized critic is trained using the value regression loss in Eq. (22).

Deployment. At execution time, the centralized critic is discarded, and all agents act using only their decentralized actors.

3.4.5. HRL: hierarchical baselines for complex behaviors

Motivation. In long-horizon and highly dynamic combat, end-to-end continuous control may suffer from insufficient exploration and long-horizon credit assignment. Planax therefore adopts a hierarchical RL (HRL) baseline that decomposes decision-making into two levels: a high-level policy selects maneuver primitives, while a low-level controller outputs continuous flight controls. This hierarchy improves both trainability and interpretability (Fig. 6).

Two time scales. Let the environment step be Δt (50 Hz in this paper). The high-level policy runs every K steps (e.g., $K=10$) and outputs a discrete mode z_k , e.g., turn, climb, evade, or pursue:

$$z_k \sim \pi_{\text{high}}(z | o_{kK}). \quad (27)$$

Conditioned on the selected mode, the low-level policy runs at the simulation rate and outputs continuous controls $\mathbf{u}_t = [\delta_a, \delta_e, \delta_r, \delta_T]$:

$$\mathbf{u}_t \sim \pi_{\text{low}}(\mathbf{u} | o_t, z_k), \quad t \in [kK, (k+1)K - 1]. \quad (28)$$

In this baseline, the high-level module follows MAPPO-style training, while the low-level module is optimized using PPO-style continuous control.

3.4.6. Self-play: training mode for Combat

Motivation. In competitive multi-agent combat, training against a fixed opponent can lead to overfitting and weak generalization. Self-play mitigates this issue by continuously adapting the opponent distribution during learning, thereby improving robustness.

Algorithm 3 Unified training loop (PPO / IPPO / MAPPO / HRL variants)

Require: Environment $\mathcal{E}$, agents $\{1, \dots, N\}$, max steps M

```

1: Initialize actors  $\{\theta_i\}$ , critic  $\phi$ , and buffer  $\mathcal{B}$ 
2: while total_steps <  $M$  do
3:   for  $t = 1$  to  $T_{\text{rollout}}$  do
4:     for each agent  $i$  in parallel do
5:       if HRL enabled then
6:         Sample mode  $z_k \sim \pi_{\text{high},i}(o_t^i)$  (every  $K$  steps)
7:         Sample  $a_t^i \sim \pi_{\text{low},i}(o_t^i, z_k)$ 
8:       else
9:         Sample  $a_t^i \sim \pi_{\theta_i}(o_t^i)$ 
10:      end if
11:      Execute  $a_t^i$  as normalized controls  $\mathbf{u}_t^i$ 
12:    end for
13:    Step  $\mathcal{E}$  and store  $(o_t, a_t, r_t, o_{t+1}, d_t)$  in  $\mathcal{B}$ 
14:  end for
15:  Compute  $\hat{A}$  and  $\hat{R}$  by GAE
16:  for each mini-batch  $b \subset \mathcal{B}$  do
17:    Update actors via Eq. (21)
18:    Update critic (centralized for MAPPO; local for PPO/IPPO)
19:  end for
20: end while

```

Training protocol. During self-play, each side is controlled by policies sampled from the same population. Opponents can be (i) the latest policy snapshot or (ii) concurrently learning counterparts. This setting naturally produces a curriculum of increasingly stronger opponents.

Evaluation. Performance is summarized in two complementary ways: (i) *internal* self-play performance within the training population, and (ii) *external* generalization against a fixed baseline (vs baseline), reported using mission-level metrics (e.g., win rate, survivors, total return).

3.4.7. Unified training and evaluation loop (consistent with the workflow)

Workflow consistency. The baseline library follows the closed-loop workflow in Fig. 5: batched rollout, periodic testing, and mission-level reporting. Algorithm 3 formalizes a unified execution template for IPPO/MAPPO (including HRL variants).

4. Benchmarking Study and Results

Section overview. This section reports quantitative benchmarking results of the proposed platform, including (i) simulator throughput and memory scalability, (ii) learning performance on the benchmark tasks, and (iii) ablation studies that isolate the impact of key design factors.

4.1. Experimental Setup

Hardware and software. All experiments are conducted on NVIDIA A100 GPUs (80 GB). Unless otherwise stated, each configuration is repeated with multiple random seeds, and results are reported as mean $\pm$ standard error.

Rollout configuration and horizon. Following the benchmark definition in Section 3.3, the default simulation step is $\Delta t = 0.02$ s. For *evaluation*, each episode lasts 200 s, i.e., $T_{\text{eval}} = 10,000$ steps.

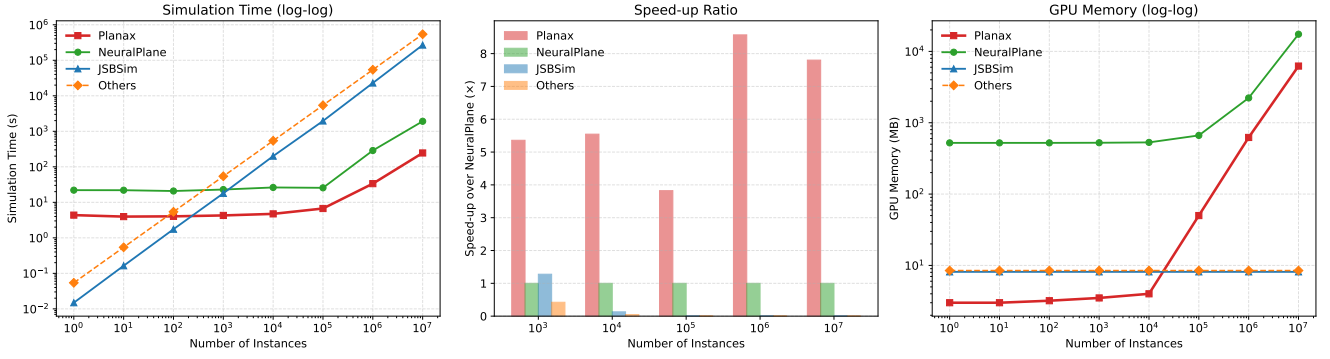


Fig. 7 Performance comparison between Planax and several classic platforms. **Left:** Relationship between the time required to simulate 10 seconds and the number of parallel simulations. **Middle:** Speed-up factor of each platform relative to NeuralPlane. **Right:** Relationship between GPU memory usage for simulating 10 seconds and the number of parallel simulations. **Red:** Results for Planax. **Green:** Results for NeuralPlane²⁵. **Blue:** Results for JSBSim³. **Orange:** Results for other platforms^{4,26}.

For *on-policy training*, policy updates are performed on truncated rollouts: the default setting uses $n = 300$ parallel rollouts per iteration and $m = 1000$ steps per rollout, corresponding to $m\Delta t = 20$ s of physical time per update. Unless otherwise specified, evaluation is conducted on full-length episodes (200 s) using the same termination constraints.

Evaluation protocol. Unless otherwise specified, periodic evaluation is performed under fixed scenario settings (e.g., fixed team size, formation template, and opponent setting), and the final performance is summarized using the same mission-level score sheets defined in Section 3.3. The training–testing–evaluation workflow follows Fig. 5.

4.2. Platform Performance and Scalability

Throughput and memory scaling. To evaluate scalability, each simulator runs 10 seconds of physical time across varying numbers of aircraft. The wall-clock time and GPU memory usage are recorded to characterize end-to-end simulation efficiency, as summarized in Fig. 7.

Key observations. The results indicate that the proposed platform achieves the fastest simulation speed across scales, maintaining sublinear growth of wall-clock time up to large parallel populations, while keeping GPU memory usage significantly lower than prior platforms at the same scale. These trends support using the platform as a practical testbed for large-scale multi-agent learning under physically grounded flight dynamics.

4.3. Validation of Low-Level Control Capabilities

Control performance analysis. Single-Control tasks are used as a sanity check for continuous flight control under the high-fidelity 6-DOF model. Fig. 8 compares learning curves for Single-Control (Heading) and the more demanding Single-Control (Ultimate Goal).

Result summary. As shown in Fig. 8, both tasks achieve steady reward improvement. Single-Control (Heading) converges rapidly, indicating that the PPO baseline can stabilize the nonlinear dynamics. Single-Control (Ultimate Goal) converges more slowly due to sustained high-bank maneuvering, but reaches strong final performance. These results confirm that Planax supports diverse control objectives without modifying the underlying dynamics engine, providing a reliable foundation for subsequent multi-agent benchmarks.

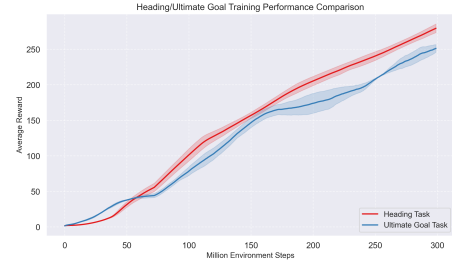


Fig. 8 Learning curves for Single-Control tasks (Heading vs. Ultimate Goal). Shaded bands depict mean $\pm$ SE over five seeds. The Heading task (red) converges faster due to lower maneuvering requirements, while the Ultimate Goal task (blue) requires learning aggressive bank-angle maneuvers.

4.4. Learning Performance on Cooperative Formation

Training stability across team sizes and templates. This subsection evaluates decentralized learning performance in the cooperative formation benchmark. Fig. 9 reports training curves under (a) different numbers of aircraft and (b) different formation templates (wedge/line/diamond), following the protocol in Section 3.3.4.

Result summary. The curves show how task difficulty increases with population size and how template geometry affects convergence speed and final performance.

4.5. Learning Performance on Combat

Evaluation settings. The combat benchmark is evaluated under two standard settings: (i) self-play and (ii) training versus a fixed-policy baseline. Both settings follow the unified interface and termination constraints in Section 3.3. Fig. 10 reports self-play results (top) and results against a fixed baseline (bottom).

Scalability and stability. As team size increases, learning tends to become less stable in combat, reflecting the difficulty of large-scale competitive multi-agent interaction. Self-play is generally more stable than training against a fixed baseline, suggesting that opponent diversity during training improves robustness.

4.6. Ablation Study

Ablation overview. This subsection isolates the impact of three key factors that strongly affect learning dynamics and scalability in large-scale decentralized continuous-control settings: (i) parallel

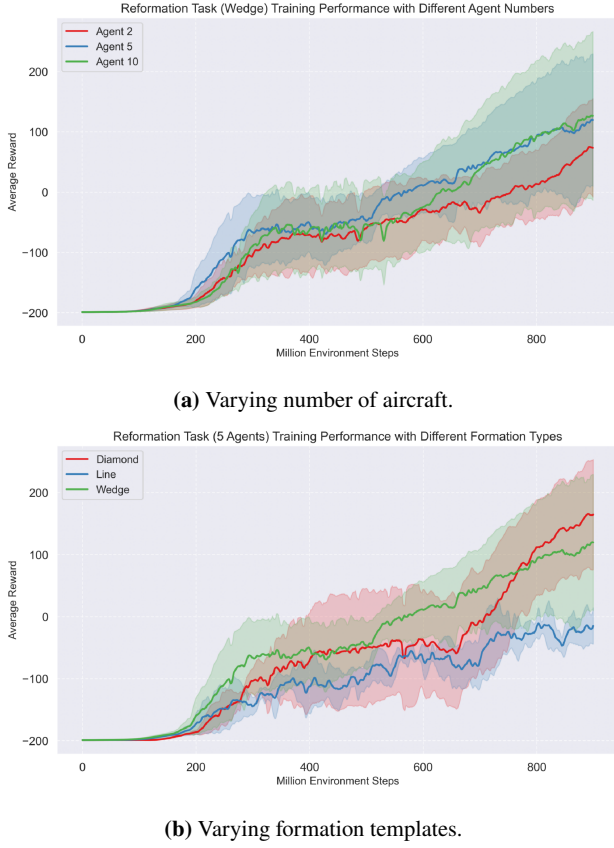


Fig. 9 Training curves of IPPO in formation task scenarios. **Top:** formation with different numbers of agents. **Bottom:** formation with different formation templates.

rollout scale, (ii) partial observability induced by range-limited information exchange, and (iii) hierarchical policy structure. Unless otherwise specified, the remaining training budgets follow Section 4.1, and the evaluation workflow follows Section 3.4.

Parallel rollout scales. To quantify the effect of data throughput on optimization stability, the number of parallel rollouts is varied across $n \in \{30, 300, 3000\}$ while keeping the scenario and policy class fixed. As the rollout scale decreases, policy optimization typically exhibits higher-variance returns and less stable improvement, whereas larger rollout scales lead to smoother and more reliable convergence. This trend highlights the role of large-batch simulation in stabilizing policy-gradient learning for long-horizon multi-agent tasks (Fig. 11, right).

Partial observability via communication distance. To study range-limited information exchange, the communication/sensing distance is varied in a self-play combat setting with a fixed team size (e.g., 5-vs-5). Three distances are considered: $d \in \{5, 10, 20\}$ km. Interestingly, enlarging the distance does not necessarily improve learning and may slightly reduce stability (Fig. 11, left). A plausible explanation is that the baseline policy does not explicitly disentangle local versus global cues; therefore, expanding the input space can increase optimization difficulty rather than providing useful structure. These results indicate that observation design can be a dominant factor in large-scale MARL performance.

Hierarchical policy structure. To evaluate whether hierarchy improves robustness in complex multi-agent scenarios, a hierarchical

policy (HRL) is compared with an end-to-end (E2E) policy under matched budgets and protocols. The comparison is conducted under multiple combat configurations (e.g., 1-vs-1, 2-vs-2, and 5-vs-5). Across settings, HRL tends to exhibit more stable learning and better final performance, whereas E2E training often struggles to converge, likely because the task requires both low-level continuous control and higher-level decision-making. These observations support decomposing behaviors into structured sub-policies for improved trainability (see Fig. 12).

Win/success rates against a fixed-policy baseline. To provide a complementary endpoint metric, the learned policies are further evaluated against a fixed-policy baseline. Table 7 reports the success (win) rates under different team sizes. The hierarchical policy achieves substantially higher success rates than the end-to-end policy across all settings. Notably, success rates against the fixed baseline tend to increase with the number of agents, suggesting that fixed strategies become less effective as coordination complexity grows.

Takeaways. Overall, the ablation results indicate that (i) increasing parallel rollout scale is a practical lever to reduce optimization variance, (ii) naively enlarging the communication range may not help unless the policy/encoder leverages the induced structure, and (iii)

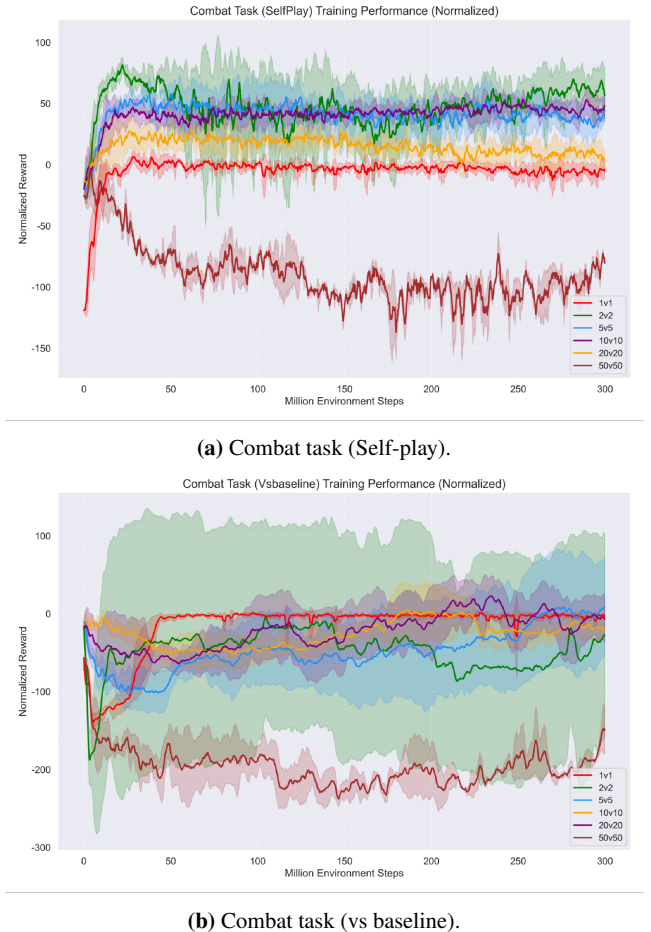


Fig. 10 Training curves of PPO in combat task scenarios. **Top:** self-play. **Bottom:** vs baseline. (HRL-MAPPO / E2E-PPO variants follow Table 6).

Table 7 Success rates of learned policies against a fixed-policy baseline in combat.

Policy type	Team size	Success rate (mean $\pm$ SE)
Self-play (Hier) vs baseline	1-vs-1	0.446 $\pm$ 0.062
Self-play (Hier) vs baseline	2-vs-2	0.653 $\pm$ 0.112
Self-play (Hier) vs baseline	5-vs-5	0.864 $\pm$ 0.131
Self-play (E2E) vs baseline	1-vs-1	0.062 $\pm$ 0.081
Self-play (E2E) vs baseline	2-vs-2	0 $\pm$ 0
Self-play (E2E) vs baseline	5-vs-5	0 $\pm$ 0

hierarchical decomposition can substantially improve convergence and robustness in multi-agent combat settings.

4.7. Discussion

Key observations from benchmarking results. The benchmarking study in Section 4 yields three consistent observations. First, the platform achieves substantially higher rollout throughput and better memory scalability than conventional “CPU-simulator + GPU-policy” pipelines, enabling stable on-policy training at much larger parallel scales. Second, the evaluated tasks exhibit distinct sources of instability as the scale increases: Single-Control is mainly limited by long-horizon credit assignment under strict flight-envelope constraints, Formation becomes sensitive to partial observability

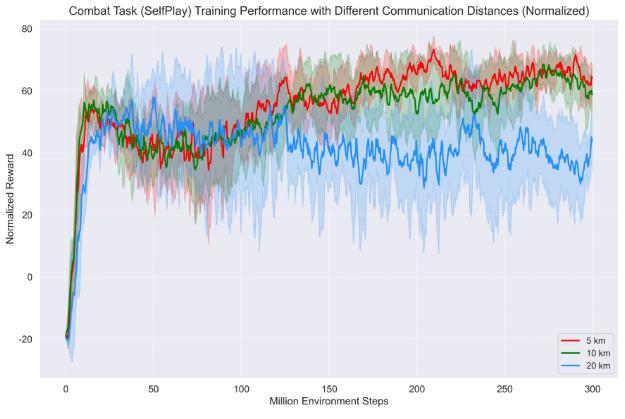
and neighbor-selection radius, whereas Combat further amplifies non-stationarity due to opponent adaptation. Third, architectural choices noticeably affect learning dynamics in challenging regimes: centralized-training–decentralized-execution improves combat stability, and the hierarchical baseline provides a more reliable optimization path in long-horizon, high-dynamic settings.

Implications for large-scale aerial MARL. These results indicate that scalable benchmarking should explicitly control and report three axes: (i) rollout scale (parallel environments and horizon length), (ii) information constraints (communication/sensing range and neighbor set size), and (iii) non-stationarity sources (opponent diversity and self-play protocol). The presented benchmark suite therefore provides a practical basis for systematic studies on population scaling behavior, self-play robustness, and structure-aware policy designs (e.g., hierarchy), under physically grounded fixed-wing dynamics.

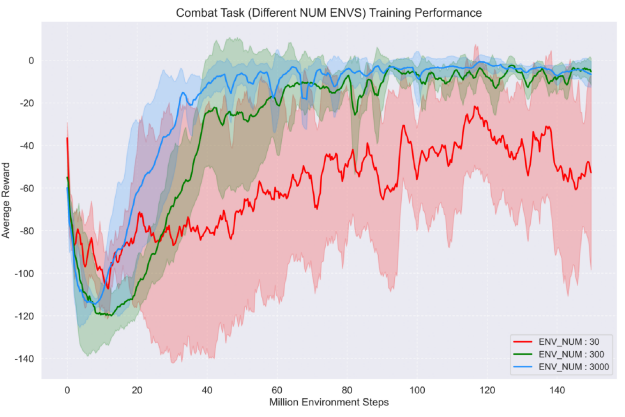
5. Conclusions

This study presents Planax, a JAX/XLA-accelerated high-fidelity parallel simulation framework for large-scale fixed-wing UAV swarm MARL. Planax integrates a physics-consistent 6-DOF model into device-resident compiled rollouts and provides a standardized benchmark suite covering single-control, formation, and combat. Experiments demonstrate scalability up to 10^8 parallel agents and more than $10\times$ throughput improvement over NeuralPlane, while retaining favorable memory scaling (Fig. 7).

Future work will extend Planax toward broader high-fidelity aircraft dynamics and mission sets. Planned directions include richer aerodynamic/propulsion models and multiple airframe configurations, long-horizon single-aircraft aerobatic maneuvering, and large-scale multi-UAV cooperative and combat tasks with more realistic sensing/communication and engagement logic.



(a) Training curves under different communication/sensing distances (5/10/20 km) in a fixed-size self-play combat setting.



(b) Comparative learning dynamics under different parallel rollout scales ($n = 30/300/3000$).

Fig. 11 Ablations on rollout scale and partial observability.

End-to-End SelfPlay Combat Task Performance

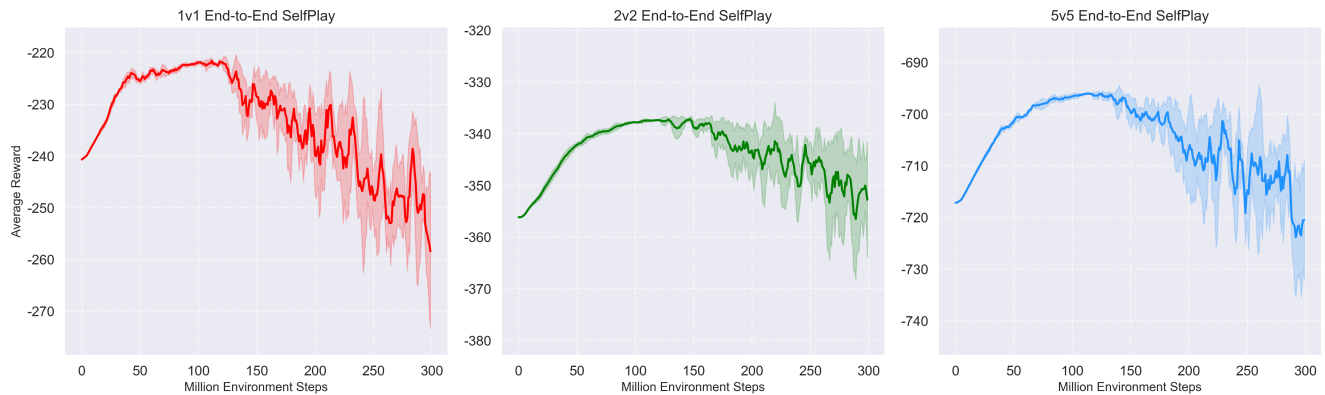


Fig. 12 Effect of hierarchical policy structure in self-play combat. HRL typically yields more stable learning dynamics and stronger final performance than end-to-end policies under matched protocols and budgets (shown for 1-vs-1, 2-vs-2, and 5-vs-5 settings).

References

- Yu C, et al. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. In: *Advances in Neural Information Processing Systems*; 2022. p. 24611-24.
- Sunehag P, Lever G, Gruslys A, et al. Value-Decomposition Networks for Cooperative Multi-Agent Learning. In: *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*; 2018. p. 2085-7.
- Berndt JS. JSBSim: An Open Source Flight Dynamics Model in C++. In: *AIAA Modeling and Simulation Technologies Conference*; 2004. p. 4923.
- Laminar Research. X-Plane Flight Simulator; 2025. <https://www.x-plane.com/>.
- Schulman J, Wolski F, Dhariwal P, et al. Proximal Policy Optimization Algorithms; 2017.
- Bohn E, et al. Deep Reinforcement Learning Attitude Control of Fixed-Wing UAVs Using Proximal Policy Optimization. In: *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE; 2019. p. 543-8.
- Bohn E, et al. Data-Driven Runway Localization for Fixed Wing Aircraft Using Deep Reinforcement Learning. *Aerospace Science and Technology* 2023;133:108109.
- Zhen Y, Hao T. Deep Reinforcement Learning Attitude Control of Fixed-Wing UAVs. In: *2020 3rd International Conference on Unmanned Systems (ICUS)*. IEEE; 2020. p. 239-44.
- Li S, Xue C, Ma X, et al. Basic Maneuver Discovery for Fixed-Wing Aircraft via Reinforcement Learning. *Aerospace Science and Technology* 2023;140:108412.
- Quessy Y, et al. Automating Fixed Wing Forced Landings with Offline Reinforcement Learning. In: *Proceedings of the International Micro Air Vehicle Conference and Competition (IMAV)*; 2023. p. 1-8.
- Lee G, Kim K. A Controllable Agent by Subgoals in Path Planning Using Goal-Conditioned Reinforcement Learning. *IEEE Access* 2023;11:33812-25.
- Lee G, Kim K, Jang J. Real-Time Path Planning of Controllable UAV by Subgoals Using Goal-Conditioned Reinforcement Learning. *Applied Soft Computing* 2023;146:110660.
- Fu Z, et al. Memory-Based Reinforcement Learning for Fly-by-Wire Flight Control. In: *Proceedings of the International Conference on Automation and Robotics (ICAR)*; 2022. p. 1-6.
- Pope A, et al. Hierarchical Reinforcement Learning for Air Combat. *IEEE Transactions on Artificial Intelligence* 2022;3(6):878-89.
- Li Y, et al. Reinforcement Learning for Formation Flight Control: A Review. *Aerospace Science and Technology* 2023;134:108135.
- Rashid T, Samvelyan M, De Witt CS, et al. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In: *Proceedings of the 35th International Conference on Machine Learning (ICML)*; 2018. p. 4295-304.
- Rashid T, Farquhar G, Peng B, et al. Weighted QMIX: Expanding Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In: *Advances in Neural Information Processing Systems*; 2020. p. 10199-210.
- Jiang Y, et al. Learning Agile Fighter Maneuvers Using Hierarchical Multi-Agent Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems* 2024. Early Access.
- Lowe R, et al. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In: *Advances in Neural Information Processing Systems*; 2017. .
- Samvelyan M, et al. The StarCraft Multi-Agent Challenge. In: *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*; 2019. p. 2186-8.
- Zhou Y, et al. Cooperative decision-making algorithm for UCAV formation. *Chinese Journal of Aeronautics* 2024;37(8):311-28.
- Brittain M, et al. Autonomous Air Traffic Controller: A Deep Multi-Agent Reinforcement Learning Approach; 2019.
- Zhao Q, et al. Research Progress of Reinforcement Learning for Fixed-Wing Aircraft Control. *Flight Dynamics* 2020;38(5):1-14.
- Richter C, et al. QPlane: An Open-Source Reinforcement Learning Toolkit for Autonomous Fixed Wing Aircraft Simulation. In: *Proceedings of the ACM Symposium on Applied Computing (SAC)*; 2021. p. 1065-72.
- Xue C, et al. NeuralPlane: An Efficiently Parallelizable Platform for Fixed-Wing Aircraft. In: *Advances in Neural Information Processing Systems*; 2024. .
- ArduPilot Development Team. ArduPilot: Open Source Autopilot Software; 2025. <https://ardupilot.org/>.
- NASA. XPlaneConnect: An Open Source Interface to the X-Plane Flight Simulator; 2016. <https://github.com/nasa/XPlaneConnect>.
- Liu Q, et al. Light Aircraft Gym: A Lightweight and Extensible Gym Environment; 2022. <https://github.com/Improbable-AI/light-aircraft-gym>.
- Makoviychuk V, et al. Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning; 2021.
- Bradbury J, et al. JAX: Composable Transformations of Python+NumPy Programs; 2018. <https://github.com/google/jax>.
- Freeman D, et al. Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation; 2021.
- Rutherford A, et al. JaxMARL: A Suite of Multi-Agent Reinforcement Learning Environments and Algorithms in JAX; 2023.
- Stevens BL, Lewis FL, Johnson EN. *Aircraft Control and Simulation*. 3rd ed. John Wiley & Sons; 2015.

Appendix A. Supplementary Expressions for the High-Fidelity Model

Appendix A.1. DCM $[T_{BE}]$ from quaternion

The direction cosine matrix from Earth/NED to body frame is:

$$[T_{BE}] = \begin{bmatrix} q_0^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y + q_0 q_z) & 2(q_x q_z - q_0 q_y) \\ 2(q_x q_y - q_0 q_z) & q_0^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z + q_0 q_x) \\ 2(q_x q_z + q_0 q_y) & 2(q_y q_z - q_0 q_x) & q_0^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix} \quad \mathbf{F}_B^{(T)} = \begin{bmatrix} T \\ 0 \\ 0 \end{bmatrix}, \quad T = f(\delta_T, h, M). \quad (\text{A1}) \quad (\text{A9})$$

Weight in body frame is obtained as

$$\mathbf{W}_B = [T_{BE}] [0, 0, mg]^\top. \quad (\text{A2})$$

Appendix A.2. Quaternion kinematics matrix

For $\boldsymbol{\omega} = [p, q, r]^\top$ and $\mathbf{q} = [q_0, q_x, q_y, q_z]^\top$,

$$\boldsymbol{\Omega}(\boldsymbol{\omega}) = \begin{bmatrix} 0 & -p & -q & -r \\ p & 0 & r & -q \\ q & -r & 0 & p \\ r & q & -p & 0 \end{bmatrix}, \quad \dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega}(\boldsymbol{\omega}) \mathbf{q}. \quad (\text{A3})$$

Appendix A.3. Component-wise 6-DOF equations (expanded form)

For completeness, the component form corresponding to Eq. (3) is:

$$\dot{u} = rv - qw + \frac{1}{m}(W_x + F_x^{(A)} + F_x^{(T)}), \quad (\text{A4a})$$

$$\dot{v} = -ru + pw + \frac{1}{m}(W_y + F_y^{(A)} + F_y^{(T)}), \quad (\text{A4b})$$

$$\dot{w} = qu - pv + \frac{1}{m}(W_z + F_z^{(A)} + F_z^{(T)}), \quad (\text{A4c})$$

and the standard inertia-coupled rotational dynamics can be written using the compact rigid-body form in Eq. (3b), or equivalently:

$$\dot{p} = (C_1 r + C_2 p)q + C_3 L_b + C_4 N_b, \quad (\text{A5a})$$

$$\dot{q} = C_5 pr - C_6(p^2 - r^2) + C_7 M_b, \quad (\text{A5b})$$

$$\dot{r} = (C_8 p - C_2 r)q + C_4 L_b + C_9 N_b, \quad (\text{A5c})$$

where $[L_b, M_b, N_b]^\top = \mathbf{M}_B$ are body-frame moments.

Appendix A.4. Aerodynamic transformations and force/moment expressions

The wind-to-body transformation depends on (α, β) :

$$[T_{BW}] = \begin{bmatrix} \cos \alpha \cos \beta & -\cos \alpha \sin \beta & -\sin \alpha \\ \sin \beta & \cos \beta & 0 \\ \sin \alpha \cos \beta & -\sin \alpha \sin \beta & \cos \alpha \end{bmatrix}. \quad (\text{A6})$$

Given dynamic pressure $\bar{q} = \frac{1}{2} \rho V^2$,

$$\begin{bmatrix} L \\ D \\ Y \end{bmatrix} = \bar{q} S \begin{bmatrix} C_L \\ C_D \\ C_Y \end{bmatrix}, \quad \mathbf{F}_B^{(A)} = [T_{BW}]^\top \begin{bmatrix} -D \\ -Y \\ -L \end{bmatrix}. \quad (\text{A7})$$

The body-frame aerodynamic moment is

$$\mathbf{M}_B^{(A)} = \begin{bmatrix} L_b \\ M_b \\ N_b \end{bmatrix} = \bar{q} S \begin{bmatrix} b C_l \\ \bar{c} C_m \\ b C_n \end{bmatrix}. \quad (\text{A8})$$

Appendix A.5. Thrust model (interface form)

Planax uses a thrust mapping $T = f(\delta_T, h, M)$ and applies it along the body longitudinal axis (with optional thrust-line offsets if modeled):

$$\mathbf{F}_B^{(T)} = \begin{bmatrix} T \\ 0 \\ 0 \end{bmatrix}, \quad T = f(\delta_T, h, M). \quad (\text{A9})$$

If a thrust-line offset is included, the corresponding moment can be added as $\mathbf{M}_B^{(T)} = \mathbf{r}_T \times \mathbf{F}_B^{(T)}$.

Appendix B. Detailed Derivations of Inertia Tensor Coefficients

The nonlinear rotational dynamics of fixed-wing aircraft involve complex gyroscopic and inertial coupling. To ensure the clarity of the main text, this appendix provides the expanded definitions of the coefficients $C_1 \dots C_9$ used in Eq. (A5). These coefficients are derived from the inertia tensor components ($I_{xx}, I_{yy}, I_{zz}, I_{xz}$) according to the standard aeronautical derivations in³³.

First, define the determinant-like term Γ :

$$\Gamma = I_{xx} I_{zz} - I_{xz}^2 \quad (\text{B1})$$

Then, the nine coefficients are computed as:

$$\begin{aligned} C_1 &= \frac{(I_{yy} - I_{zz})I_{zz} - I_{xz}^2}{\Gamma}, & C_2 &= \frac{(I_{xx} - I_{yy} + I_{zz})I_{xz}}{\Gamma} \\ C_3 &= \frac{I_{zz}}{\Gamma}, & C_4 &= \frac{I_{xz}}{\Gamma}, & C_5 &= \frac{I_{zz} - I_{xx}}{I_{yy}} \\ C_6 &= \frac{I_{xz}}{I_{yy}}, & C_7 &= \frac{1}{I_{yy}}, & C_8 &= \frac{(I_{xx} - I_{yy})I_{xx} + I_{xz}^2}{\Gamma}, & C_9 &= \frac{I_{xx}}{\Gamma} \end{aligned} \quad (\text{B2})$$

These coefficients facilitate the calculation of angular accelerations ($\dot{p}, \dot{q}, \dot{r}$) in the fused GPU kernel, preserving bit-for-bit consistency with validated F-16 flight models^{25,33}.

Appendix C. Aerodynamic Force Transformations and Scalar-Vector Clarification

Appendix C.1. Distinction between Scalar q and Quaternion $\mathbf{q}$

In the kinematics and dynamics formulations (Eqs. 3–8), it is critical to distinguish between two frequently used symbols:

1. **Scalar q :** A component of the angular rate vector $\boldsymbol{\omega} = [p, q, r]^\top$, representing the *pitch rate* of the aircraft in the body frame $\mathcal{F}_B$.
2. **Vector $\mathbf{q}$:** The attitude quaternion $\mathbf{q} = [q_0, q_x, q_y, q_z]^\top$ (or $[q_0, q_1, q_2, q_3]^\top$), which provides a global *attitude representation* relative to the Earth frame $\mathcal{F}_E$ and is integrated using the skew-symmetric matrix $\boldsymbol{\Omega}(\boldsymbol{\omega})$.

Appendix C.2. Aerodynamic Force Calculation

Aerodynamic forces—Lift (L), Drag (D), and Sideforce (Y)—are defined in the Wind Frame $\mathcal{F}_W$ based on the airflow velocity. The transformation to the Body Frame $\mathcal{F}_B$ is performed via the matrix $[T_{BW}]$, which depends on the angle of attack α and sideslip angle β ³³:

$$[T_{BW}] = \begin{bmatrix} \cos \alpha \cos \beta & -\cos \alpha \sin \beta & -\sin \alpha \\ \sin \beta & \cos \beta & 0 \\ \sin \alpha \cos \beta & -\sin \alpha \sin \beta & \cos \alpha \end{bmatrix} \quad (\text{C1})$$

Table D.8 Experimental parameter settings for different task scenarios.

Task Name	Rollouts (n)	Steps/Rollout (m)	Total Steps (M)	Scenario	Model
Heading	300	1000	3×10^8	Heading Task	F-16
Ultimate Goal	300	1000	3×10^8	Continuous Turn	F-16
Formation	300	1000	3×10^8	Formation Task	F-16
Combat	300	1000	3×10^8	Combat Task	F-16

The final aerodynamic force vector in $\mathcal{F}_B$ is obtained as:

$$[F_x^{(A)}, F_y^{(A)}, F_z^{(A)}]^\top = [T_{BW}]^\top [-D, -Y, -L]^\top \quad (\text{C2})$$

The magnitudes are derived using the high-fidelity neural surrogate models trained on NASA F-16 data^{25,33}:

$$\begin{aligned} L &= \frac{1}{2} \rho V^2 S C_L, & C_L &= \text{MLP}_{C_L}(\alpha, \beta, \delta), \\ D &= \frac{1}{2} \rho V^2 S C_D, & C_D &= \text{MLP}_{C_D}(\alpha, \beta, \delta), \\ Y &= \frac{1}{2} \rho V^2 S C_Y, & C_Y &= \text{MLP}_{C_Y}(\alpha, \beta, \delta). \end{aligned} \quad (\text{C3})$$

where ρ is the air density, V is the airspeed, S is the reference wing area, and δ denotes control inputs. As discussed in Section 3.2.4, this branch-free MLP formulation helps maintain more uniform control flow during massively parallel GPU rollouts, and is generally friendly to JAX/XLA compilation^{25,30}.

Appendix D. Implementation and Experimental Details

Appendix D.1. Experimental Hyperparameters

To ensure reproducibility, the specific parameter configurations for the simulation environment and training schedules are detailed in Table D.8. All tasks utilize the high-fidelity F-16 dynamics model.

Appendix D.2. Software Architecture

To facilitate extensibility, Planax adopts a modular architecture. As illustrated in Fig. D.13, the platform exposes a single unified RL interface, while encapsulating complex dynamics and logic within four backbone modules:

- Simulation Environment (Orange Block):** The JAX-based flight dynamics engine advances up to 10^8 aircraft in parallel. It computes vectorised observations, rewards, and termination flags, returning them to the RL loop efficiently.
- Task Scenarios (Green Block):** Each scenario specifies the task-specific logic, including reward functions, termination conditions, and initial state distributions. Planax currently natively supports the Formation and Combat tasks described in Section 3.3.
- Baseline Library (Blue Block):** This module integrates reference learners, including perception-optimised IPPO for formation tasks, and hierarchical MAPPO/End-to-End PPO for combat.
- Evaluation Tools (Purple Block):** This module handles data logging and visualization. It records mission-level statistics and exports high-resolution trajectory files compatible with Tacview for qualitative replay.

Appendix D.3. Execution Routines

Based on the architecture above, Planax implements automated routines for training and evaluation. Algorithm 4 and Algorithm 5 detail the execution logic for the Trainer and Evaluator modules, respectively.

Algorithm 4 Planax Trainer Routine

Require: Learnable agent A , Flight Dynamics Model M , Task T

- 1: $Env \leftarrow \text{InitialiseEnv}(M, T)$
- 2: **while** total_steps < step_max **do**
- 3: $A.\text{train_episode}(Env)$
- 4: Log rollout data and render real-time curves
- 5: **end while**
- 6: **return** trained A and full training log

Algorithm 5 Planax Evaluator Routine

Require: Algorithm set $\mathcal{B}$, Model M , Task T

- 1: $Env \leftarrow \text{InitialiseEnv}(M, T)$
- 2: **for all** $alg \in \mathcal{B}$ **do**
- 3: $alg.\text{rollout_episode}(Env)$
- 4: Record trajectory files (.csv/.txt) and mission scores
- 5: **end for**
- 6: Generate comparative tables, radar charts, and Tacview replay files

Appendix E. Supplementary Results

Appendix E.1. Cross-Scale Transferability

A key capability of Planax is enabling policies trained on small-scale scenarios to generalize to larger swarms. Transferability is evaluated by training on small teams ($N_{train} \in \{2, 5, 10\}$) and testing on significantly larger teams ($N_{test} \in \{25, 50, 100\}$).

Table E.9 presents the zero-shot transfer performance. Results indicate that policies trained on medium-sized groups (e.g., 10 agents) generalize effectively to 100 agents, maintaining high success rates. This validates that the local observation formulation in Planax effectively captures scalable coordination rules.

Appendix F. Used Assets and Licenses

Planax is an open-source tool available at <https://anonymous.4open.science/r/Aeroplanax-2087>. Planax strictly adheres to the open-source licenses of the assets and libraries integrated into the framework. Table E.10 lists the key components and their respective licenses.

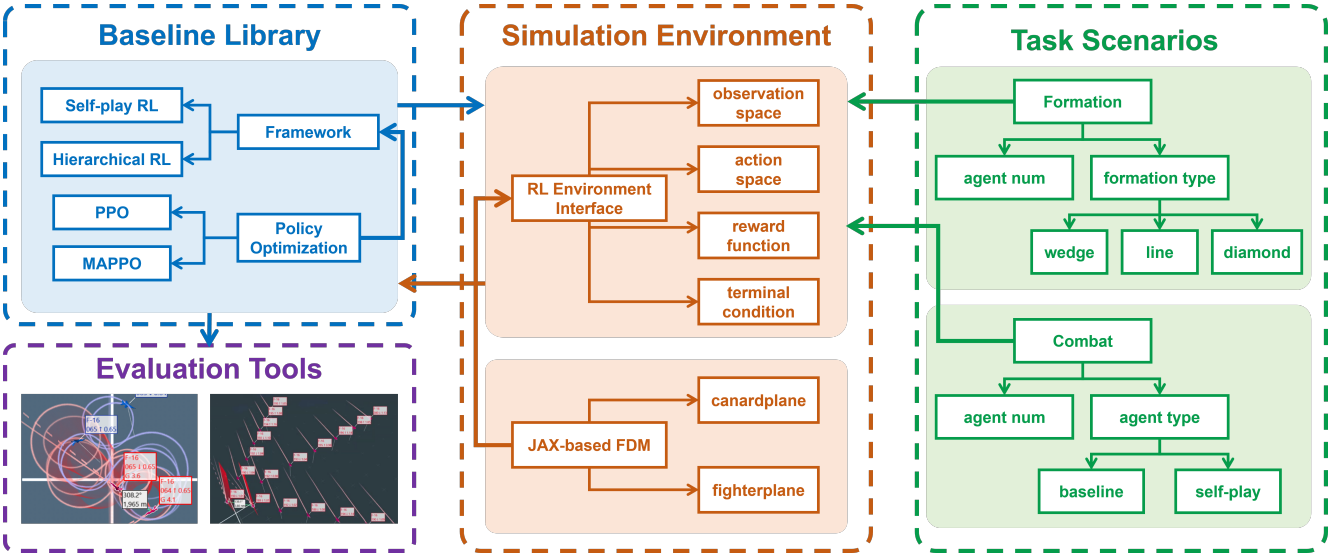


Fig. D.13 Overall software architecture of Planax. Arrows denote data flow during each simulation step. The modular design decouples the learning algorithms from the high-fidelity physics engine.

Table E.9 Cross-scale transfer performance in the Formation task (Mean $\pm$ SE).

Test Size	Train Size	Success Rate	Total Reward	Survivors	Sim Steps
25	2	0.666 $\pm$ 0.008	2.10 $\pm$ 3.33	9.91 $\pm$ 0.01	1125 $\pm$ 12
	5	0.977 $\pm$ 0.002	35.98 $\pm$ 0.47	9.97 $\pm$ 0.01	1000 $\pm$ 0.1
	10	0.980 $\pm$ 0.002	51.52 $\pm$ 0.34	9.98 $\pm$ 0.01	1000 $\pm$ 0.0
50	2	0.517 $\pm$ 0.010	-28.65 $\pm$ 4.62	49.45 $\pm$ 0.02	1237 $\pm$ 16
	5	0.876 $\pm$ 0.004	35.52 $\pm$ 0.34	49.47 $\pm$ 0.01	1001 $\pm$ 0.6
	10	0.901 $\pm$ 0.004	51.75 $\pm$ 0.17	49.89 $\pm$ 0.01	1000 $\pm$ 0.0

Table E.10 Assets employed in Planax and their licenses.

Category	Asset / Codebase	License
Baseline Algs	JaxMARL / PureJaxRL	Apache-2.0
RL Skeleton	LAG (Light Aircraft Game)	LGPL-3.0
Comparison Platform	JSBSim 1.1	LGPL-2.1
Comparison Platform	XPlane	Commercial